第 1 回目　IoT デバイス Airoco の基本

1.1 IoT デバイス Airoco の基本

IoT の技術を学ぶために，換気状態のセンサ Airoco を用いる．Airoco デバイスの機能，クラウド上でのデータ蓄積，JavaScript と Python での WebAPI を使った情報取得と表示，ブラウザのデベロッパーツールの使用方法について学習する．

1.1.1 IoT とは

IoT とは，「Internet of Things」の略であり，様々なモノがインターネットに接続され情報交換を行うことで相互に制御する仕組みである．従来はパソコンやサーバーなどの IT 関連機器がインターネットに接続されていたが，コンピュータの小型化，低価格化，インターネットの高速化，無線通信技術の進歩などにより，多くのモノがネット接続されるようになった．現在では，スマートフォンやタブレット端末，スマートウォッチ，カメラ，デジタル録画機，スマートスピーカーなど様々な機器がネットに接続され，この流れは今後もますます増えていくと考えられる．インターネットにつながるモノのことを IoT デバイスと呼ぶ．

1.1.2 IoT のデータフォーマット

IoT では，ネットワークに繋がったたくさんのセンサが収集したデータはクラウドなどに送られ，蓄積・分析される．インターネットでデータを送るにはあらかじめ送るデータのフォーマットを決めておく必要がある．実際に使われているフォーマットは以下の3つが代表的であり，それぞれに特徴がある．

- CSV (Comma Separated Values)：カンマ形式とも呼ばれ，古くから利用されているデータ形式である．各データをカンマで区切って表現し，1 行が 1 件分のデータを表す．カンマで区切られた順番がデータ項目を表すので，送信側と受信側でデータ項目の並び順が合っている必要がある．また，途中のデータが存在しない場合でも，必ずカンマを記述して，該当位置まで認識させる必要がある．そのため，データ項目の変更が多かったり，扱う種類が多かったりする IoT での使用にはあまり向いていない．
- XML (Extensible Markup Language)：共通の基本構文を用い，言語活用を用意にしたマークアップ言語である．「<」と「>」で囲まれたタグと呼ばれる情報を表記することで，データに何らかの意味を付加して利用する．これにより，送信側と受信側でタグの項目名を決めておけば，CSV 形式とは異なり並び順を意識する必要がなくなる．また，不要なカンマを位置合わせのためにたくさん記述しなければならなかった問題点も解消される．ただし，すべてのデータ項目をタグで囲う必要があるので，データサイズが大きくなる．
- JSON (Javascript Object Notation)：Web で利用される JavaScript のオブジェクト表記をベースにした形式である，キーと実際のデータをセットにした軽量なデータ記述方法で，システム間やプログラム間でデータの受け渡しに利用されている．キーが XML 形式のタグと同様の働きをするので，表記順を意識する必要がなく，無駄なカンマも不要である．さらに，XML 形式ではデータ終了部分にタグで項目名を再度表記するが，JSON 形式ではカンマでデータの区切りを表しているので，データサイズを小さくすることができる．

1.1.3 WebAPI について

API はアプリケーション・プログラミング・インタフェース（Application Programming Interface）の略で，ソフトウェアコンポーネント同士が互いに情報をやりとりするためのインタフェースの仕様である．IoT では，HTTP や HTTPS などの Web 技術を利用して，ネットワーク上にあるプログラムの提供する機能を別のプログラムで利用できるようにしている．この仕組みが WebAPI であり，代表的なものとして以下の 2 種類がある．

- SOAP (Simple Object Access Protocol) API： XML 形式のデータでリクエストとレスポンスを行う．SOAP の URL は操作と対応付けられている．
- REST (REpresentational State Transfer) API： ネットワーク上のコンテンツを一意の URL で表し，その URL に HTTP が持つメソッド（GET, POST, PUT, DELETE）でリクエストを行う．レスポンスは XML や JSON などで受け取る．現在主流になっている WebAPI である．

参考： IoT 検定ユーザー教育推進ワーキンググループ，IoT のしくみと技術がしっかりわかる教科書，技術評論社，2020.

1.2 Airoco について

1.2.1 Airoco の概要

Airoco（エアロコ）は，中部電力が開発した換気状態を見える化するための IoT デバイス，および，サービスである．ただし，サービスの運営は 2025 年 4 月 1 日より合同会社ネコリコに変更となっている．デバイスからは二酸化炭素濃度に加え，温度，湿度を取得可能であり，データはクラウド上に蓄積される．蓄積されたデータは Web ページとして表示したり，WebAPI を使って取得したりすることができる．Airoco の形状を Fig. 1 に，設置イメージを Fig. 2 に示す．

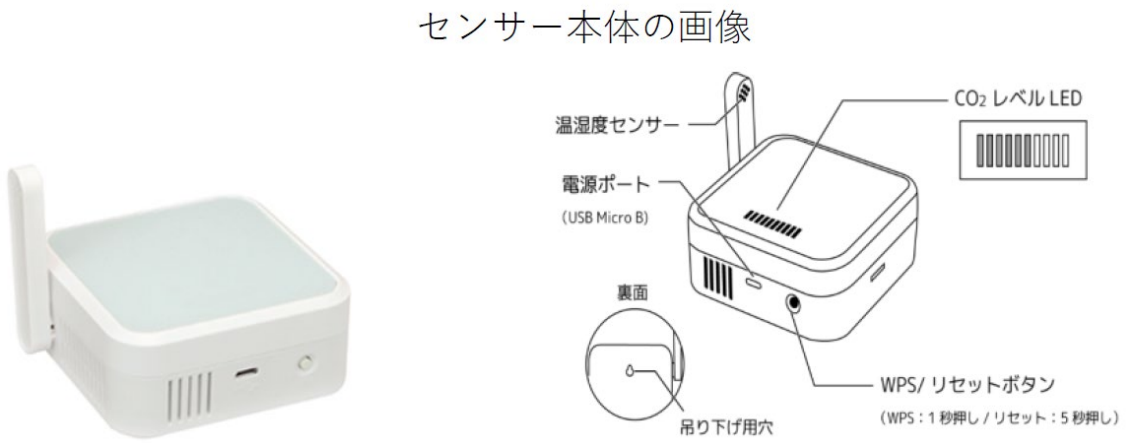


Fig. 1 Airoco の形状

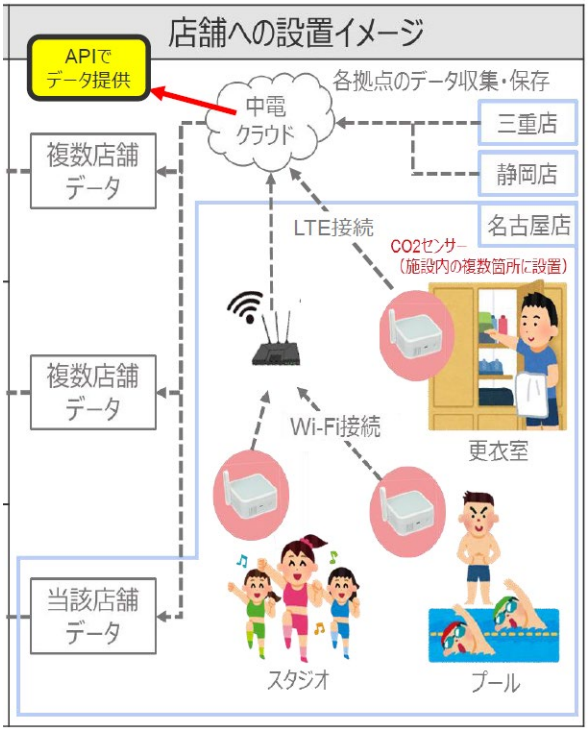


Fig. 2　Airoco の設置イメージ（中部電力資料より）

1.2.2 Airoco のセンサの動作原理

1.2.2.1 CO2 センサ

経産省が 2021 年 11 月に発表した「二酸化炭素濃度測定器の選定等に関するガイドライン」(https://www.meti.go.jp/covid-19/guideline.pdf)で，換気が十分行われているかを判断するための CO2 検知原理として「光学式」であることを推奨している．光学式は，ガス特有の吸収波長領域を利用して計測する方式である．Fig. 3 に示すようにそれぞれのガスで光を吸収する波長が違い，CO2 分子は 4.26 μm を吸収する．

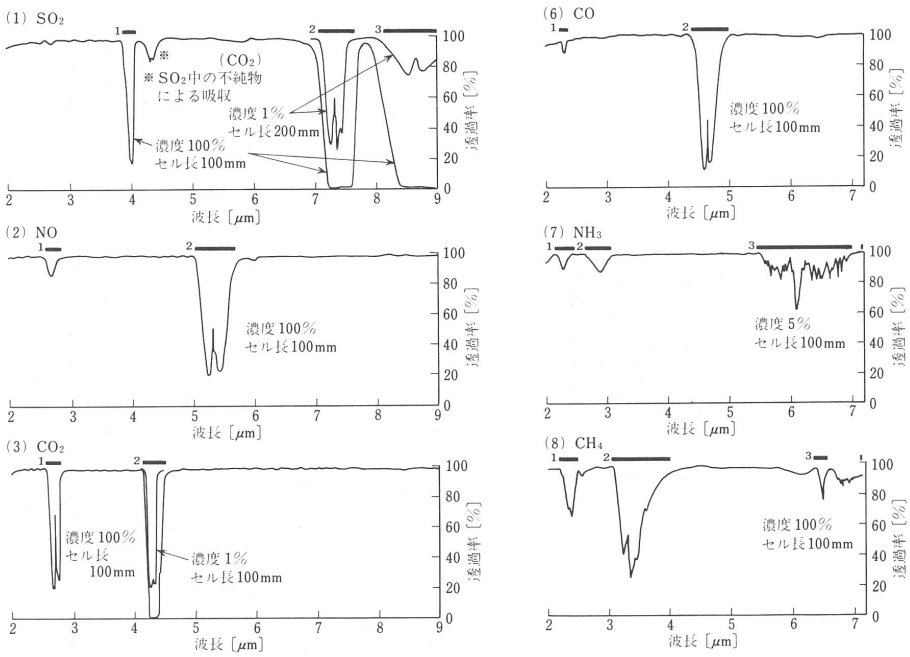


Fig. 3　気体の赤外線吸収スペクトル（山崎弘郎，センサ工学の基礎　第 2 版，昭晃堂，2000　より）

光学式には NDIR(Non Dispersive InfraRed)方式と PhotoAcoustic（光音響）方式があり，Airoco は PhotoAcoustic 方式を採用している．PhotoAcoustic 方式は，光エネルギーを吸収して起こる熱膨張によって発生した音響波をマイクロフォンで検出する仕組みであり，小型化に優れている．PhotoAcoustic 方式では，IR エミッターから CO2 分子の吸収域と一致する，周期的に断続したパルス状の狭帯域光を完全密閉した測定セル内に放射する．CO2 が入射光を吸収すると温度が上がり入射光の周期で音響波が発生する．音響波の大きさは CO2 濃度に依存するので，音響波をマイクロフォンで検出することで二酸化炭素濃度が取得できる．

1.2.2.2 温度・湿度センサ

Airoco には温度・湿度センサも搭載されている．これには Sensirion の IC である SHTC3 を利用している．主要な特性を以下に示す．より詳細が知りたい場合は，データシートを参照する．
https://www.sensirion.com/media/documents/643F9C8E/63A5A436/Datasheet_SHTC3.pdf

温度：

温度精度 0.2℃

作業温度範囲 −40～+125℃

反応時間 5～30s

湿度：

相対湿度精度 2%RH

作業相対湿度範囲 0～100%RH

反応時間 8s

1.2.1.3 Airoco の設置場所

現在，Airoco は

・R3-401（センサ名「R3ー401」）
・R3-403（センサ名「R3ー403」）
・R3-301 (innovatoin hub) （センサ名「R3ー301」）
・R3-4F エレベーターホール（センサ名「R3ー4F_EH」）
・R3-3F エレベーターホール（センサ名「R3ー3F_EH」）
・R3-1F エレベーターホール（センサ名「R3ー1F_EH」）
・R3-B1F エレベーターホール（センサ名「R3ーB1F_EH」）

に設置されている．

1.3 Airoco が集めた情報へのアクセス方法

1.3.1 ネコリコ提供サイトの表示

各センサで集めた情報はネットワークを通してネコリコが管理するサイトに集められる．この情報はネコリコ提供サイトで Web ページとして見ることができるし，WebAPI を使った情報取得もできる．

Web ページを表示するには，ネコリコから伝えられた以下のサイトにアクセスする．

■施設管理者向け画面

https://airoco.necolico.jp/#/?hash=CgETViZ2

Fig. 4　Airoco サイトの施設管理者向け画面

施設管理者向け画面を Fig. 4 に示す．各センサ部分をクリックすることで特定の期間のグラフを表示したり，右上の歯車マークをクリックすることで CSV ファイルをダウンロードしたりなどができる．

1.3.2　JavaScript による WebAPI へのアクセス

クラウドに集められた Airoco のセンサデータを取得するために WebAPI が使用できる．　Airoco の WebAPI にアクセスするための情報は「センサデータ API 仕様　3.0 版」に記述されている．以下に概略を示す．

ネコリコから以下の情報が提供されている．

・API URL

https://airoco.necolico.jp

・サブスクリプションキー

6b8aa7133ece423c836c38af01c59880

・ID　ハッシュキー

CgETViZ2

最新の全センサデータを JSON で取得するには，以下のアドレスにアクセスする

https://airoco.necolico.jp/data-api/latest?id=CgETViZ2&subscription-key=6b8aa7133ece423c836c38af01c59880

24 時間分の全センサデータを CSV で取得するには，以下のアドレスにアクセスする．

https://airoco.necolico.jp/data-api/day-csv?id=CgETViZ2&subscription-key=6b8aa7133ece423c836c38af01c59880&startDate=1687359600

startDate の値は，UNIX 時間で示した開始日時である．例の値は，2023 年 6 月 22 日 0 時 0 分 0 秒を表している．

この WebAPI は REST API と呼ばれるタイプで，URL に対し GET でリクエストを送信し，JSON 形式や CSV ファイルでレスポンスを受け取る．

1.4 Airoco の WebAPI にアクセスするプログラム

1.4.1 JavaScrip による最新のセンサ情報表示

最新のセンサ情報を WebAPI で取得して特定のセンサについて表示する JavaScript プログラムが sample1-1.html である．sample1-1.css と同一ディレクトリに置いて使用する．サーバーと HTTP 通信をするために，JavaScript では XMLHttpRequest（略称 XHR）が使われてきた．近年ではより完結に表現できる FetchAPI が使われるようになってきているが，ここでは XMLHttpRequest を用いた例を見る．XMLHttpRequest は Microsoft によるいわゆる独自実装の API として始まり，後追いで W3C/WHATWG により標準化された．現在では XMLHttpRequest を直接利用する機会は減っており，fetch 関数などを利用するようになってきているが，それらの技術を理解する上でも XMLHttpRequest は重要な基本となる．

WebAPI を叩く通信は，処理時間を自分でコントロールすることができず長くなる場合がある．そこで止まってしまうのを避けるため，JavaScript で用意されている XMLHttpRequest や fetch では非同期処理が行われる．非同期処理とは，1 つ目の処理の完了を待たずに 2 つ目の処理実行を開始する処理を指す．実行の順番を待たないので，処理を完了する順番はその都度異なることになる．これに対し，同期処理は 1 つ目の処理完了後に 2 つ目の処理を開始する．

JavaScript で通信に同期処理を行うと，表示画面が更新されなくなることがある．なぜなら，通信に時間がかかっている場合，続く処理が行われないので，読み込みが完了するまでページ内の JavaScript をブロックするからである．Web ブラウザによっては，スクロールなどができなくなる．

以下のプログラム sample1-1(Fig. 5)をダウンロードし，Web ブラウザで実行する．

```
1   <!DOCTYPE html>
2   <html>
3
4   <head>
5       <meta charset="UTF-8">
6       <meta name="viewport" content="width=device-width">
7       <meta http-equiv="refresh" content="150; URL=">
8       <link rel="stylesheet" type="text/css" href="sample1-1.css">
9       <script>
10          var request = new XMLHttpRequest();
11          request.open('GET', 'https://airoco.necolico.jp/data-api/latest?id=CgETViZ2&subscr
12          request.responseType = 'json';
13          request.send();
14          request.onload = function () {
15              var sens_num = 1; // センサ番号
16              var jsonData = this.response;
17              var data = JSON.stringify(jsonData, null, ' ');
18              jsons = JSON.parse(data);
19              console.log(jsons);
20              console.log(jsons[sens_num].co2);
21              var cmnt = "---";
22              var obj = document.getElementById("co2");
23              if (jsons[sens_num].co2 >= 1000) {
24                  obj.style.backgroundColor = "#EE5500";
25                  cmnt = "<hr><p align = \"center\" class = \"txt4\">やや混雑</p>";
26              } else {
27                  obj.style.backgroundColor = "#00AA00";
28                  cmnt = "<hr><p align = \"center\" class = \"txt4\">換気良好</p>";
29              }
30              co2.innerHTML = "<p align = \"center\" class = \"txt1\"><br></p><p align =\"ce
31          }
32          request.onerror = function () {
33              console.log("受信失敗");
34          }
35      </script>
36  </head>
37
38  <body>
39      <p class="txt4" align="center">当部屋のCO2濃度は</p>
40      <div id="co2"></div>
41      <p class="txt4" align="center">情報工学科</p>
42  </body>
43
44  </html>
```

Fig. 5　sample1-1

簡単に内容を説明する.

L10: XMLHttpRequest オブジェクト request を作成,

L11: XMLHttpRequest.open():リクエストの初期化. 第1引数で, ”GET”か”POST”のメソッド指定. 第 2 引数は URL, 第3引数は同期, 非同期の指定であり, true で非同期(デフォルト値)となる.

L12: XMLHttpRequest.responseType: レスポンスのデータ型を指定

L13: XMLHttpRequest.send():リクエストをサーバーに送信する.

L14: XMLHttpRequest.onload: コンテンツのフェッチ成功時に呼び出されるハンドラ. ここに, load イベントを受信したときに呼び出す関数を記述する.

非同期通信を行うので, 受信時のハンドラを定義して, 受信後に呼び出されるようにしている. load 時に呼び出される関数内では, JSON をパースし, 取り出したセンサ機器の CO2 値を Web ページで表示している.

Web ブラウザには, HTML, CSS, JavaScript などの動作を確認するための開発者ツールが組み込まれている. ここでは, Google Chrome を想定して, 開発者ツールであるデベロッパーツール(DevTools)の使い方を学習する.

sample1-1.html では, console.log()でデータを表示している. これをデベロッパーツールで確認する(Fig. 6). Chrome 右上の3点リーダーアイコンをクリックして出るメニューから, 「その他のツール」→「デベロッパーツール」を選択し, 表示

する．コンソールタブを選択して，console.log の出力が表示されることを確認する．JSON のデータは最初は折りたたまれて表示されるので，左端三角形をクリックして展開して中を確認する．

Fig. 6　デベロッパーツール

1.4.2　JavaScript の Fetch を使ったセンサ情報表示

sample1-2 (Fig. 7)をダウンロードして，Web ブラウザで実行する．この例では HTML ファイルとは別に JavaScript プログラムを書いた sample1-2.js を用意している．また，グラフを表示(Fig. 8)するために，Chart.js ライブラリを利用している．html ファイル内で以下の記述により，ネットワーク上から Chart.js の機能を組み込んでいる．

<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.2.0/chart.min.js"></script>

```
6   async function getCsvGraph() {
7     var date = new Date();     // Dateオブジェクトを作成
8     var a = date.getTime();    // UNIXタイムスタンプを取得する（ミリ秒単位）
9     var curr_time = Math.floor( a / 1000 );  // UNIXタイムスタンプを取得する（秒単位）
10    var tt = curr_time - 3600*24   // 24時間前
11    const res = await fetch(`https://airoco.necolico.jp/data-api/day-csv?id=CgETViZ2
12    console.log("aaa");
13    const raw_data = await res.text();
14    console.log(raw_data);
15    const data = convertArray(raw_data);
16    for (var row of data){
17      row[3] -= curr_time;
18    }
```

Fig. 7　sample1-2

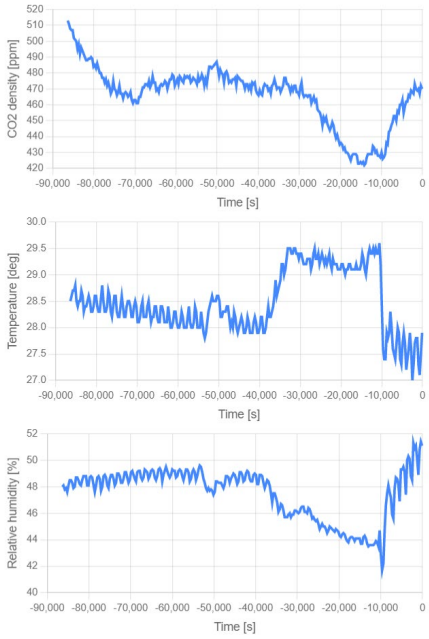


Fig. 8　JavaScript によるグラフ表示

このプログラムでは，JavaScript で HTTP 通信を行うために，Fetch API を用いる．XMLHttpRequest よりシンプルでモダンな非同期処理が書ける．fetch 関数では，呼び出し先の URL を記載する．今回は，開始時刻を指定し，24 時間分のデータが CSV 形式で得られる API を使用している．非同期処理を指示するため，await をつけていること，関数全体に async が付いていることに注意する．具体的には，getCsvGraph()の前に async が，fetch 関数とその結果のテキストへの変換の前に await がついている．

await は async function の内部でのみ使用することができ，非同期処理の完了の結果を待つために使用する．await がついた関数は Promise オブジェクトを返す．Promise オブジェクトは履行されるか拒否されるまで実行を中断することで，Promise を返す関数をあたかも同期しているかのように動作させる．fetch 関数の場合は，HTTP リクエストの結果が返されるまで待つ．Promise の解決済みの値は，await 式の返り値として扱われる．

参考資料:

- XMLHttpRequest のマニュアル

  https://developer.mozilla.org/ja/docs/Web/API/XMLHttpRequest/responseType

- Fetch API のマニュアル

  https://developer.mozilla.org/ja/docs/Web/API/Fetch_API

- Chart.js のホームページ

  https://www.chartjs.org/docs/latest/

- セットアップガイド（CO2 センサー Wi-fi モデル）.pdf （最終ページに，構造や再起動の情報がある）
- センサデータ API 仕様_v3.0.pdf
- async, await について

  https://masanyon.com/javascript-promise-async-await-%E9%9D%9E%E5%90%8C%E6%9C%9F%E5%87%A6%E7%90%86/

1.4.3 Python による最新のデータ取得

Python を使って Airoco の WebAPI から情報を取得するサンプル sample1-3.ipynb (Fig. 9)を示す. この例では, Python の実行環境として, Google Colaboratory を使用する. サンプルプログラムを各自の Google ドライブにアップロードすれば実行可能である. 各自の PC に Python3 の実行環境が用意されている場合は, それを用いても良い.

```
import requests
import datetime

res = requests.get('https://airoco.necolico.jp/data-api/latest?id=CgETViZ2&subscription-key=6b8aa7133ece423c836c
# print(res.status_code)
# print(res.text)
for i in range(9):
  res_json = res.json()[i]
  # print(res_json)
  sensorNumber = res_json['sensorNumber']
  sensorName = res_json['sensorName']
  print(sensorNumber, sensorName)
  co2 = res_json['co2']
  if co2==None: continue # データ無ければ次
  temperature = res_json['temperature']
  relativeHumidity = res_json['relativeHumidity']
  timestamp = res_json['timestamp']
  dt = datetime.datetime.fromtimestamp(timestamp)
  print(f'CO2濃度={co2}[ppm], 気温={temperature}[deg], 相対湿度={relativeHumidity}[%], タイムスタンプ={dt}¥n');
```

Fig. 9 sample1-3.ipynb

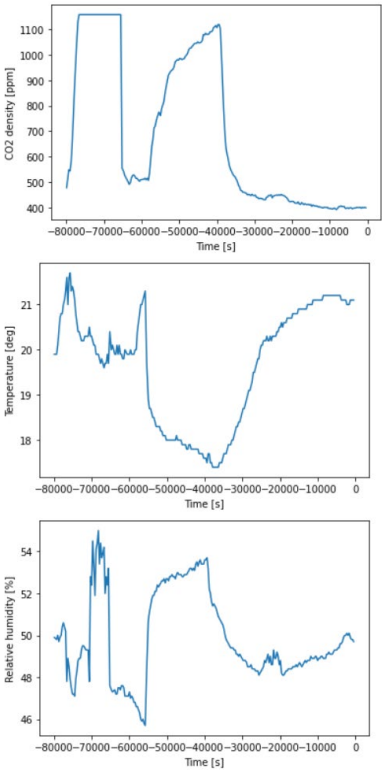
Python で WebAPI にアクセスするために, Requests モジュールを使用する(Colab では既にインストールされている.) requests.get 関数で, GET メソッドを実行できる. なお, JavaScript と違って Web ブラウザの動きを止める心配がないので, このモジュールは非同期ではない.

1.4.4 Python による時間推移データのグラフ表示

Python でグラフを表示(Fig. 10)するために, sample1-4.ipynb を実行する. このプログラムではグラフ表示のために Matplotlib を用いる. これも Google Colaboratory に既にインストールされているので, 自分でインストールの必要はない(別の処理系を用いるときには必要になることもある).

CSV でデータを得るので, その解析に csv モジュールを使用する(標準ライブラリなので追加でインストール不要).

サンプルのグラフの横軸は, 現在から遡った秒数となっている.



参考：

Matplotlib の HP　https://matplotlib.org/

1.5 課題

基本課題は全員が実行する．発展課題は，時間があれば実行する．

【基本課題 1-1】ネコリコ提供サイトから，R3-B1F エレベーターホールの Airoco センサの直近 2 日の CO2 濃度推移を表すグラフを取得し，なぜそのようなグラフになったかを考察せよ．

【基本課題 1-2】sample1-2 の getCsvGraph()中の fetch 直後と，getCsvGraph()呼び出しの次の行に console.log への出力を記述し，実行時にどちらが先に実行されているか確認せよ．また，どうしてそのような結果になるか考察せよ．

【基本課題 1-3】どれか1台の Airoco を選択し，Python で過去 1 週間分の CO2 濃度推移をグラフ表示するプログラムを作成せよ．

【発展課題 1-1】「課題 1-3」で作ったプログラムの横軸を日付表示にせよ．

【発展課題 1-2】2 台以上の Airoco センサから得られる出力（CO2 濃度，温度，湿度）を組み合わせてわかりやすく画面表示する JavaScript プログラムを作成せよ．（2 台以上で）何台にするか，最新データのみを用いるか時間推移を表示するかは各自の方針に従って自由に決めて良い．